

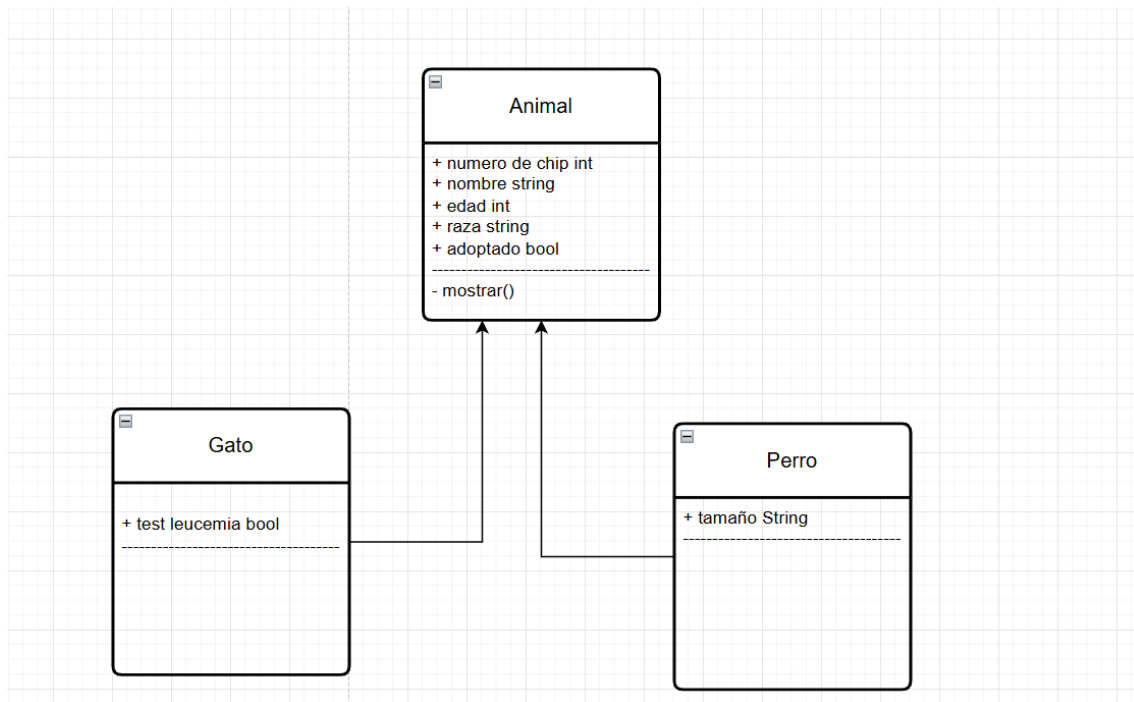
HITO 1 DEL 3º TRIMESTRE DE PROGRAMACIÓN

Alejandro Cortés Díaz

Índice

Diagrama de clases.....	2
Definicion de clases.....	2
Funcionamiento código.....	10
Enlace a GitHub	13

Diagrama de clases



Definicion de clases

Animal;

```
1 //Creo la clase abstracta animal de la que saldrán sus hijas, con un constructor que contiene prácticamente todos los atributos de las otras dos clases, y un método que posteriormente, cada clase tendrá que sobrescribir con sus propios datos
2 public abstract class Animal {
3
4     int numero_chip;
5     String nombre;
6     int edad;
7     String raza;
8     boolean adoptado;
9
10    public Animal (int numero_chip, String nombre, int edad, String raza, boolean adoptado) {
11        this.numero_chip= numero_chip;
12        this.nombre = nombre;
13        this.edad = edad;
14        this.raza = raza;
15        this.adoptado= adoptado;
16    }
17
18    abstract void mostrar();
19 }
20
21
```

//Creo la clase abstracta animal de la que saldrán sus hijas, con un constructor que contiene prácticamente todos los atributos de las otras dos clases, y un método que posteriormente, cada clase tendrá que sobrescribir con sus propios datos

```

public abstract class Animal {

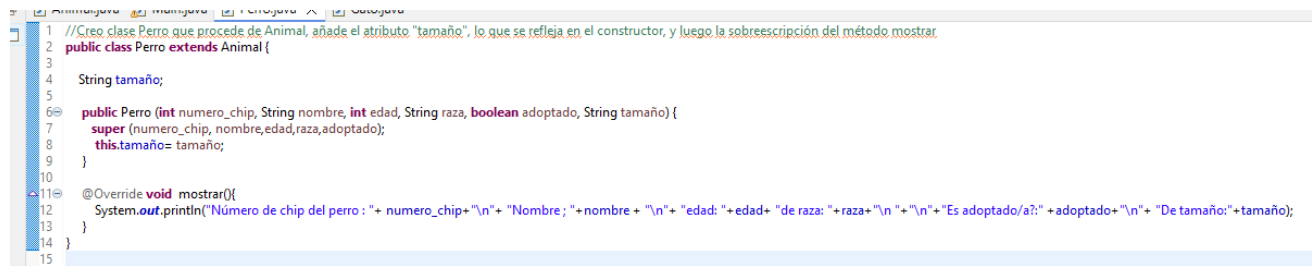
    int numero_chip;
    String nombre;
    int edad;
    String raza;
    boolean adoptado;

    public Animal (int numero_chip, String nombre, int edad, String raza, boolean adoptado) {
        this.numero_chip= numero_chip;
        this.nombre = nombre;
        this.edad = edad;
        this.raza = raza;
        this.adoptado= adoptado;
    }

    abstract void mostrar();
}

```

Perro;



```

1 //Creo clase Perro que procede de Animal, añade el atributo "tamaño", lo que se refleja en el constructor, y luego la sobreescritción del método mostrar.
2 public class Perro extends Animal {
3
4     String tamaño;
5
6     public Perro (int numero_chip, String nombre, int edad, String raza, boolean adoptado, String tamaño) {
7         super (numero_chip, nombre,edad,raza,adoptado);
8         this.tamaño= tamaño;
9     }
10
11     @Override void mostrar(){
12         System.out.println("Número de chip del perro : "+ numero_chip+"\n" + "Nombre: "+ nombre + "\n" + "edad: "+edad+ " de raza: "+raza+"\n " +"\n" + "Es adoptado/a?:" + adoptado+"\n" + "De tamaño:" +tamaño);
13     }
14 }
15

```

//Creo clase Perro que procede de Animal, añade el atributo "tamaño", lo que se refleja en el constructor, y luego la sobreescritción del método mostrar

```

public class Perro extends Animal {

    String tamaño;

    public Perro (int numero_chip, String nombre, int edad, String raza, boolean adoptado, String tamaño) {
        super (numero_chip, nombre,edad,raza,adoptado);
        this.tamaño= tamaño;
    }

    @Override void mostrar(){
        System.out.println("Número de chip del perro : "+ numero_chip+"\n" + "Nombre ;
"+nombre + "\n" + "edad: "+edad+ " de raza: "+raza+"\n " +"\n" + "Es adoptado/a?:" + adoptado+"\n" + "De
tamaño:" +tamaño);
    }
}

```

Gato;

```
1 //Creo clase Perro que procede de Animal, añade el atributo "test_leucemia", lo que se refleja en el constructor, y luego la sobrescripción del método mostrar
2
3 public class Gato extends Animal{
4
5     boolean test_leucemia;
6
7     public Gato (int numero_chip, String nombre, int edad, String raza, boolean adoptado, boolean test_leucemia) {
8         super (numero_chip, nombre, edad, raza, adoptado);
9         this.test_leucemia= test_leucemia;
10    }
11    @Override void mostrar(){
12        System.out.println("Número de chip del gato : " + numero_chip+"\n" + "Nombre: " + nombre + "\n" + "edad: " + edad + "\n" + "de raza: " + raza + "\n " + "\n" + "Es adoptado/a?: " + adoptado + "\n");
13    }
14 }
15
```

//Creo clase Perro que procede de Animal, añade el atributo "test_leucemia", lo que se refleja en el constructor, y luego la sobrescripción del método mostrar

```
public class Gato extends Animal{

    boolean test_leucemia;

    public Gato (int numero_chip, String nombre, int edad, String raza, boolean adoptado, boolean
test_leucemia) {
        super (numero_chip, nombre, edad, raza, adoptado);
        this.test_leucemia= test_leucemia;
    }
    @Override void mostrar(){
        System.out.println("Número de chip del gato : " + numero_chip+"\n" + "Nombre ;
"+nombre + "\n" + "edad: " + edad + "\n" + "de raza: " + raza + "\n " + "\n" + "Es adoptado/a?: " + adoptado + "\n");
    }
}
```

Main;

```
1 import java.util.*; // Importo los utilís de java
2
3 public class Main {
4
5     Scanner sc = new Scanner(System.in); // Creo la instancia del scanner
6
7     ArrayList<Animal> animals = new ArrayList<>(); // Y el arraylist donde depositaré los objetos de la clase padre Animal
8
9     public static void main(String[] args) { // En main creo la instancia de la clase para realizar las operaciones de los métodos
10
11         Main hito1 = new Main();
12         hito1.menu();
13     }
14
15     public void menu () {
16
17         try { // Try catch para añadir una corrección de errores en el caso de que no funcione correctamente o de error (aplica el mismo comentario en los otros try catch)
18
19             int opcion;
20
21             do { // Me aseguro de que al menos una vez muestre todas las opciones pues será necesario para interactuar con el programa, hasta que la opción seleccionada sea la 4, salir
22
23                 System.out.println("Posibles elecciones: \n -----");
24
25                 System.out.println("1 - Agregar perro \n 2 - Agregar gato \n 3 - Buscar animal \n - 4 Salir del programa");
26
27                 opcion = sc.nextInt();
28
29                 // Cada caso es una opción dentro del programa, en la tercera a través del scanner pido el chip y llamo a la función para buscar según el chip encontrado dentro del array. La cuarta opción permite salir del bucle while, mientras que si s
30                 switch(opcion) {
31
32                     case 1:
33
34                         añadirPerro();
35
36                         break;
37
38                     case 2:
39
40                         añadirGato();
41                         break;
42                     case 3:
43
44                         System.out.println("Dame el número de chip para mostrar sus datos ");
45                         int chip = sc.nextInt();
46
47                         mostrarAnimals(chip);
48                         break;
49                     case 4:
50
51                         System.out.println("Saliendo del programa...");
52                         break;
53                     default:
54                         System.out.println("No es opcion válida");
55                 }
56                 System.out.println("La elección anterior fue "+opcion);
57             }
58
59             while (opcion != 4);
60
61
62
63
64
65
66         } catch (Exception e) {
67             System.out.println("Excepcion" + e);
68
69         }
70     }
71
72     // En esta función (dentro de catch try por los posibles errores), establezco que el número de chip sea el tamaño del array +1, lo que evita repeticiones de id, así como que sea una adición incremental de este valor.
73     // Poco a poco se piden los valores de los atributos correspondientes que se añadirán a la nueva instancia de perro con los valores coincidentes del constructor
74     public void añadirPerro() {
75
76         try {
77
78
79             int num_chip = numAnimales()+1;
80
81             sc.nextLine();
82
83             System.out.println("Introduce nombre.");
84
85             String nombre = sc.nextLine();
86
87             System.out.println("Introduce edad.");
88             int edad = sc.nextInt();
89
90             sc.nextLine();
91
92             System.out.println("Introduce raza.");
93             String raza = sc.nextLine();
94
95             System.out.println("es adoptado?.");
96             boolean adoptado = sc.nextBoolean();
97
98             sc.nextLine();
99
100             System.out.println("Introduce tamaño.");
```

```
System.out.println("Introduce tamaño.");
String tamaño = sc.nextLine();

animals.add(new Perro(num_chip,nombre,edad,raza,adoptado,tamaño));
}catch(Exception e) {
    System.out.println("Excepcion"+e);
}

}

// En esta función (dentro de catch try por los posibles errores), establezco que el número de chip sea el tamaño del array + 1, lo que evita repeticiones de id, así como que sea una adición incremental de este valor.
// Poco a poco se piden los valores de los atributos correspondientes que se añadirán a la nueva instancia de perro con los valores coincidentes del constructor

public void añadirGato() {

    try {

        int num_chip = numAnimales()+1;
        System.out.println("Introduce nombre.");

        sc.nextLine();
        String nombre = sc.nextLine();

        System.out.println("Introduce edad.");
        int edad = sc.nextInt();

        System.out.println("Introduce raza.");
        String raza = sc.nextLine();
        sc.nextLine();

        System.out.println("¿es adoptado?");
        boolean adoptado = sc.nextBoolean();

        System.out.println("Tiene test leucemia?");
        boolean test_leucemia = sc.nextBoolean();

        animals.add(new Gato(num_chip,nombre,edad,raza,adoptado,test_leucemia));
    }catch(Exception e) {
        System.out.println("Excepcion"+e);
    }

}

// Esta es la función utilizada para obtener el tamaño del arrayList, en base a lo cual marco cuál tendría que ser el valor del array. Una variable en un bucle for recorre el array, y su valor aumenta en "1" hasta que lleva al valor del tamaño del array

public int numAnimales() {

    int no = 0;
    for (Animal animal : animals) {
        no ++;
    }
    return no;
}

// Esta función utiliza un bucle for para recorrer el array, en busca del mismo valor de chip que se introduce como parámetro, en el momento en el que el valor "numero_chip", coincide con el de "chip" pasado como parámetro,
// utiliza la función mostrar(), presente en ambas clases, para entregar los datos del animal

public void mostrarAnimals(int chip) {

    try {

        for (Animal animal : animals) {
            if (animal.numero_chip==chip) {
                animal.mostrar();

                break; }
        }

    } catch(Exception e) {
        System.out.println("Excepcion"+e);
    }

}

}
```

import java.util.*; // Importo los utils de java

public class Main {

Scanner **sc** = **new** Scanner (System.**in**); // Creo la instancia del scanner

ArrayList <Animal> **animals** = **new** ArrayList<>(); // Y el arraylist donde depositaré los objetos de la clase padre Animal

public static void main (String[]args) { // En main creo la instancia de la clase para realizar las operaciones de los métodos

Main hito1 = **new** Main();
hito1.menu();
}

public void menu () {

try { // Try catch para añadir una corrección de errores en el caso de que no funcione correctamente o de error (aplica el mismo comentario en los otros try catch)

int opcion;

do { // Me aseguro de que al menos una vez muestre todas las opciones pues será necesario para interactuar con el programa, hasta que la opción seleccionada sea la 4, salir

System.out.println("Posibles elecciones; \n -----
-----");

System.out.println("1 - Agregar perro \n 2 - Agregar gato \n - 3 Buscar animal \n - 4 Salir del programa");

opcion = sc.nextInt();

// Cada caso es una opción dentro del programa, en la tercera a través del scanner pido el chip y llama a la función para buscar según el chip encontrado dentro del array. La cuarta opción permite salir del bucle while, mientras que si se da cualquier otra se señala como no válida, puesto que no encierra función alguna.

switch(opcion) {

case 1:

añadirPerro();
break;

case 2:

añadirGato();
break;

case 3:

System.out.println("Dame el número de chip para mostrar sus datos ");

int chip = sc.nextInt();

mostrarAnimals(chip);
break;

case 4:

System.out.println("Saliendo del programa...");
break;

default:

System.out.println("No es opcion válida");

}

System.out.println("La elección anterior fue "+opcion);

}

while (opcion != 4);

}catch(Exception e) {


```

        System.out.println("Excepcion"+e);
    }
}

```

// En esta función (dentro de catch try por los posibles errores), establezco que el número de chip sea el tamaño del array +1, lo que evita repeticiones de id, así como que sea una adición incremental de este valor.

// Poco a poco se piden los valores de los atributos correspondientes que se añadirán a la nueva instancia de perro con los valores coincidentes del constructor

```

public void añadirPerro() {

    try {

        int num_chip = numAnimales()+1;

        sc.nextLine();

        System.out.println("Introduce nombre.");

        String nombre = sc.nextLine();

        System.out.println("Introduce edad.");
        int edad = sc.nextInt();

        sc.nextLine();

        System.out.println("Introduce raza.");
        String raza = sc.nextLine();

        System.out.println("es adoptado?");
        boolean adoptado = sc.nextBoolean();

        sc.nextLine();

        System.out.println("Introduce tamaño.");
        String tamaño = sc.nextLine();

        animals.add(new Perro(num_chip,nombre,edad,raza,adoptado,tamaño));
    }catch(Exception e) {
        System.out.println("Excepcion"+e);
    }

}

```

// En esta función (dentro de catch try por los posibles errores), establezco que el número de chip sea el tamaño del array +1, lo que evita repeticiones de id, así como que sea una adición incremental de este valor.

// Poco a poco se piden los valores de los atributos correspondientes que se añadirán a la nueva instancia de perro con los valores coincidentes del constructor

```

public void añadirGato() {

    try {

        int num_chip = numAnimales()+1;

```

```

        System.out.println("Introduce nombre.");

        sc.nextLine();
        String nombre = sc.nextLine();

        System.out.println("Introduce edad.");
        int edad = sc.nextInt();

        System.out.println("Introduce raza.");
        String raza = sc.nextLine();
        sc.nextLine();

        System.out.println("es adoptado?.");
        boolean adoptado = sc.nextBoolean();

        System.out.println("Tiene test leucemia?.");
        boolean test_leucemia = sc.nextBoolean();

        animals.add(new Gato(num_chip,nombre,edad,raza,adoptado,test_leucemia));
    }catch(Exception e) {
        System.out.println("Excepcion"+e);
    }
}

```

// Esta es la función utilizada para obtener el tamaño del arrayList, en base a lo cual marco cuál tendría que ser el valor del array. Una variable en un bucle for recorre el array, y su valor aumenta en "1" hasta que lleva al valor del tamaño del array

```

public int numAnimales() {

```

```

    int no = 0;
    for (Animal animal : animals) {
        no ++;
    }
    return no;

```

```

}

```

// Esta función utiliza un bucle for para recorrer el array, en busca del mismo valor de chip que se introduce como parámetro, en el momento en el que el valor "numero_chip", coincide con el de "chip" pasado como parámetro.

// utiliza la función mostrar(), presente en ambas clases, para entregar los datos del animal

```

public void mostrarAnimals(int chip) {
    try {

```

```

        for (Animal animal : animals) {
            if (animal.numero_chip==chip) {
                animal.mostrar();

                break; } }

```

```

        } catch (Exception e) {
            System.out.println("Excepcion" + e);
        }
    }
}

```

Funcionamiento código

Añadir perro

```

Posibles elecciones;
-----
1 - Agregar perro
2 - Agregar gato
- 3 Buscar animal
- 4 Salir del programa
1
Introduce nombre.
Marck
Introduce edad.
12
Introduce raza.
Bull_Dog
es adoptado?.
true
Introduce tamaño.
mediano
La elección anterior fue 1

```

Añadir gato

```

2
Introduce nombre.
Lio
Introduce edad.
24
Introduce raza.
Blanco
es adoptado?.
False
Tiene test leucemia?.
True
La elección anterior fue 2
Posibles elecciones;
-----

```

```

1 - Agregar perro
2 - Agregar gato
- 3 Buscar animal
- 4 Salir del programa

```

Buscar Animal

Busco el primero, perro

```
3
Dame el número de chip para mostrar sus datos
1
Número de chip del perro : 1
Nombre ; Marck
edad: 12de raza: Bull_Dog

Es adoptado/a?:true
De tamaño:mediano
La elección anterior fue 3
Posibles elecciones;


---


1 - Agregar perro
2 - Agregar gato
- 3 Buscar animal
- 4 Salir del programa
```

Busco el segundo, gato

```
Dame el número de chip para mostrar sus datos
2
Número de chip del gato : 2
Nombre ; Lio
edad: 24
de raza:

Es adoptado/a?:false

La elección anterior fue 3
Posibles elecciones;


---


1 - Agregar perro
2 - Agregar gato
- 3 Buscar animal
- 4 Salir del programa
```

Salir del programa

```
1 - Agregar perro
2 - Agregar gato
- 3 Buscar animal
- 4 Salir del programa
4
Saliendo del programa...
La elección anterior fue 4
```

Enlace a GitHub

<https://github.com/Cortes-cmd/Programacion.git>

